

Algoritmos Avançados

Aulas 07 & 11: Manipulação de Arquivos – Modo Texto e Modo Binário

Prof. Me. Douglas Samuel Gonçalves

Fatec Bebedouro – Jorge Caram Sabbag
Tecnologia em Big Data no Agronegócio

2º Semestre / 2026

Sumário

- 1 Introdução à Persistência de Dados
- 2 Manipulação de Arquivos em Modo Texto
- 3 Manipulação de Arquivos em Modo Binário
- 4 Comparativo: Modo Texto vs. Modo Binário
- 5 Exercícios Práticos Resolvidos

Memória Volátil vs. Armazenamento Secundário

Memória Primária (RAM)

- Acesso em alta velocidade (nanossegundos).
- Capacidade limitada e custo elevado.
- **Volátil:** todo o conteúdo é perdido quando a aplicação encerra ou há queda de energia.

Memória Secundária (SSD / HDD)

- Acesso mais lento (milissegundos / microssegundos).
- Grande capacidade de armazenamento e baixo custo relativo.
- **Não-volátil:** os dados persistem após o término da execução do algoritmo.

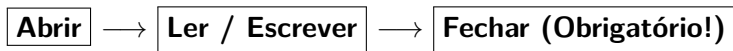
Contexto em Big Data no Agronegócio

Dados coletados em campo por sensores IoT, estações meteorológicas, drones e colheitadeiras não podem depender apenas da memória de execução; eles precisam ser armazenados e recuperados de forma íntegra e eficiente.

O Ciclo de Vida de um Arquivo

Qualquer operação sobre arquivos em sistemas operacionais modernos obedece a um ciclo rígido de 3 etapas:

- ❶ **Abertura (Open):** O sistema operacional localiza o arquivo no disco, verifica permissões de acesso e aloca um descritor de arquivo (*File Descriptor*) e um buffer na memória RAM.
- ❷ **Manipulação (Read / Write / Seek):** Fluxos de leitura ou gravação de dados operam sobre o buffer e atualizam o cursor de posição no arquivo.
- ❸ **Fechamento (Close):** Garante o descarregamento (*flush*) de dados pendentes no buffer para o disco físico e libera os recursos alocados pelo sistema operacional.



A Função Built-in `open()`

A função nativa `open()` é o ponto de entrada para qualquer manipulação de arquivos em Python.

Assinatura Principal

```
open(file, mode='r', buffering=-1, encoding=None, errors=None,
      newline=None)
```

- `file`: Caminho relativo ou absoluto para o arquivo no sistema de arquivos.
- `mode`: Modo de operação (leitura, escrita, anexação, texto ou binário).
- `encoding`: Padrão de codificação de caracteres (ex: `'utf-8'`, `'latin-1'`). Crucial no modo texto!
- Retorna um objeto do tipo **File Object** (fluxo de I/O / *stream*).

Tabela de Modos de Abertura

A combinação de letras define o comportamento exato da abertura:

Modo	Descrição	Posição Cursor	Se arquivo não existir
'r'	Leitura (modo padrão)	Início (0)	Lança FileNotFoundError
'w'	Escrita (trunca / sobrescreve)	Início (0)	Cria o arquivo
'a'	Anexação (<i>Append</i>)	Final	Cria o arquivo
'x'	Criação exclusiva para escrita	Início (0)	Lança FileExistsError
'r+'	Leitura e escrita simultânea	Início (0)	Lança FileNotFoundError
'w+'	Leitura e escrita (sobrescreve)	Início (0)	Cria o arquivo
't'	Modo Texto (padrão implícito)	—	Interpreta codificação (str)
'b'	Modo Binário	—	Manipula bytes brutos (bytes)

Buffering de I/O no Sistema Operacional

O disco é ordens de grandeza mais lento que a memória RAM. Por isso, operações de arquivo utilizam **buffers**.

Como Funciona?

- Quando chamamos `write()`, os dados vão para uma área intermediária de memória (buffer).
- Quando o buffer enche, o S.O. faz uma escrita em lote (*batch*) no disco físico.
- Reduz drasticamente as chamadas de sistema (*syscalls*).

Modos de Buffer no Python

- `buffering=0`: Sem buffer (permitido apenas em binário).
- `buffering=1`: Buffer de linha (descarrega a cada `\n` no modo texto).
- `buffering > 1`: Buffer com tamanho fixo em bytes.
- `buffering=-1`: Padrão do sistema (geralmente 4 KB ou 8 KB).

Caminhos de Arquivos e a Biblioteca pathlib

Evite concatenar caminhos com barras manuais ("pasta/arquivo.txt"). Use a biblioteca padrão pathlib para código portátil entre Windows e Linux.

```
1 from pathlib import Path
2
3 # Cria caminhos compatíveis com qualquer Sistema Operacional
4 diretorio_dados = Path("dados_campo") / "safra_2026"
5 arquivo_telemetria = diretorio_dados / "sensores.csv"
6
7 # Cria os diretorios automaticamente se nao existirem
8 diretorio_dados.mkdir(parents=True, exist_ok=True)
9
10 # Verificacoes uteis
11 print("Existe?", arquivo_telemetria.exists())
12 print("E arquivo?", arquivo_telemetria.is_file())
13 print("Extensao:", arquivo_telemetria.suffix)
```


O que é um Arquivo de Texto?

Definition (Arquivo de Texto)

Uma sequência de caracteres legíveis por humanos, codificados em bytes de acordo com uma tabela de codificação específica (ex: UTF-8, ASCII, ISO-8859-1).

Codificação (Encoding)

- Cada caractere é mapeado para 1 ou mais bytes.
- **UTF-8** é o padrão global universal (suporta acentuação, emojis e múltiplos alfabetos).
- No Windows, o padrão legado costumava ser cp1252 / latin-1.

Quebras de Linha (*Newlines*)

- Unix / Linux / macOS: `\n` (LF - Line Feed).
- Windows: `\r\n` (CRLF - Carriage Return + Line Feed).
- O Python faz a tradução universal automática de quebras de linha em modo texto.

O Gerenciador de Contexto: Cláusula with

Antigamente, usava-se `arquivo = open()` com fechamento manual via `arquivo.close()`. Se ocorresse um erro no meio, o arquivo ficava preso na memória!

Abordagem Frágil (Legada)

```
1 arq = open("dados.txt", "w")
2 arq.write("Processando...")
3 # Se ocorrer uma excecao aqui,
4 # arq.close() NUNCA sera chamado!
5 arq.close()
```

Abordagem Pythonica (with)

```
1 with open("dados.txt", "w", encoding
2           ="utf-8") as arq:
3     arq.write("Processando...")
4     # O arquivo e SEMPRE fechado
5     # automaticamente ao sair do bloco!
```

Múltiplos Arquivos no Mesmo Contexto with

Podemos abrir múltiplos fluxos de I/O em um único bloco with de forma concisa e segura:

```
1 origem = "leituras_brutas.txt"
2 destino = "leituras_limpas.txt"
3
4 # Abrindo simultaneamente o arquivo de entrada e o de saída
5 with open(origem, "r", encoding="utf-8") as f_in, open(destino, "w",
6     encoding="utf-8") as f_out:
7     for linha in f_in:
8         linha_processada = linha.strip().upper()
9         if linha_processada:
10             f_out.write(linha_processada + "\n")
```

Vantagem

Garante que ambos os arquivos sejam fechados com segurança, mesmo se ocorrer uma falha durante o processamento intermediário.

Operações de Escrita em Modo Texto

O Python oferece dois métodos principais para persistência de strings:

- `arq.write(string)`: Escreve a string informada no arquivo. ****Não insere**** quebra de linha automaticamente!
- `arq.writelines(sequencia)`: Escreve uma lista/iterável de strings em sequência (também não adiciona `\n`).

```
1 sensores = ["Temperatura: 28.5 C\n", "Umidade Solo: 64%\n", "Pressao: 1013\n", "hPa\n"]
2
3 with open("telemetria.txt", "w", encoding="utf-8") as arq:
4     # 1. Escrita individual com write()
5     arq.write("=== RELATORIO DE CAMPO ===\n")
6     arq.write("Data: 2026-09-04\n\n")
7
8     # 2. Escrita em lote com writelines()
9     arq.writelines(sensores)
```

Anexação de Conteúdo (*Append Mode*)

O modo 'a' posiciona o ponteiro de escrita no fim do arquivo, preservando dados existentes.

```
1 # Simulacao de log continuo de eventos de colheitadeira
2 def registrar_log(mensagem: str) -> None:
3     with open("eventos_colheita.log", "a", encoding="utf-8") as log:
4         log.write(f"[LOG] {mensagem}\n")
5
6 registrar_log("Motor ligado com sucesso.")
7 registrar_log("Talhao 03 iniciado. Velocidade: 6.2 km/h.")
8 registrar_log("Nivel de combustivel: 82%.")
```

Descarga Forçada de Buffer: `flush()`

O sistema operacional armazena escritas temporariamente em buffer na RAM antes de persistir no disco físico. Use `arq.flush()` para forçar a gravação imediata sem precisar fechar o arquivo.

Leitura Completa vs. Leitura Linha a Linha

read() e readlines()

- `read()`: Carrega ****todo o arquivo**** como uma única string.
- `readlines()`: Carrega todo o arquivo em uma lista de strings.
- **Perigo:** Se o arquivo tiver 10 GB, a memória RAM pode estourar!

Iterador Nativo de Linhas

- O objeto do arquivo é um **iterador sob demanda**.
- Carrega na memória apenas ****uma linha por vez****.
- Uso de memória constante: $O(1)$ independente do tamanho do arquivo.

```
1 # Leitura Eficiente para Big Data (Memory Friendly)
2 with open("grandes_dados.csv", "r", encoding="utf-8") as arq:
3     for linha in arq:
4         linha_limpa = linha.strip() # Remove o \n final
5         # Processa a linha individualmente sem pesar na RAM...
```

O Método readline() e Laços de Leitura

O método readline() lê uma única linha do arquivo por chamada, retornando uma string vazia () ao atingir o final do arquivo (EOF).

```
1 with open("configuracoes.txt", "r", encoding="utf-8") as arq:
2     # 1. Le o cabeçalho isoladamente
3     cabecalho = arq.readline().strip()
4     print("Cabeçalho:", cabecalho)
5
6     # 2. Continua lendo as próximas linhas sob demanda
7     while True:
8         linha = arq.readline()
9         if not linha: # String vazia indica EOF (End of File)
10             break
11         print("Parametro:", linha.strip())
```

Controle de Posição do Cursor: `tell()` e `seek()`

O cursor de leitura/escrita indica o deslocamento (*offset*) em bytes a partir do início.

- `arq.tell()`: Retorna a posição atual do cursor (em bytes).
- `arq.seek(offset, whence=0)`: Move o cursor para uma posição especificada.
 - `whence=0`: a partir do início do arquivo (padrão).
 - `whence=1`: a partir da posição atual do cursor.
 - `whence=2`: a partir do final do arquivo.

```
1 with open("amostra.txt", "w+", encoding="utf-8") as arq:
2     arq.write("ABCDEFGH IJ")
3     print("Posicao apos escrita:", arq.tell())    # 10 bytes
4
5     # Volta o cursor para o inicio do arquivo
6     arq.seek(0)
7     print("Leitura do inicio:", arq.read(4))    # 'ABCD'
8     print("Posicao atual:", arq.tell())          # 4
```


Tratamento Robusto de Exceções de I/O

Operações em disco estão sujeitas a falhas externas que devem ser capturadas preventivamente.

```
1 nome_arquivo = "leituras_ph.txt"
2
3 try:
4     with open(nome_arquivo, "r", encoding="utf-8") as arq:
5         conteudo = arq.read()
6 except FileNotFoundError:
7     print(f"Erro: O arquivo '{nome_arquivo}' nao foi encontrado no caminho especificado.")
8 except PermissionError:
9     print(f"Erro: Permissao negada para ler o arquivo '{nome_arquivo}'.")
10 except UnicodeDecodeError:
11     print("Erro: O arquivo nao esta codificado no padrao UTF-8 esperado.")
12 except OSError as erro:
13     print(f"Erro inesperado do Sistema Operacional: {erro}")
```

Estruturas em Modo Texto: Módulo csv

O módulo csv padroniza a persistência tabular sem necessidade de parsing manual.

Gravação (csv.writer)

```
1 import csv
2
3 dados = [
4     ["Talhao", "Cultura", "Prod"],
5     ["T-01", "Soja", 74.5],
6     ["T-02", "Milho", 110.2]
7 ]
8
9 with open("prod.csv", "w", newline=""
10 , encoding="utf-8") as f:
11     w = csv.writer(f, delimiter=";")
12     w.writerows(dados)
```

Leitura Estruturada (DictReader)

```
1 import csv
2
3 with open("prod.csv", "r", encoding="
4     utf-8") as f:
5     r = csv.DictReader(f, delimiter="
6     ;")
7     for row in r:
8         t = row["Talhao"]
9         p = row["Prod"]
10        print(f"{t}: {p} sc/ha")
```

CSV Avançado: DictWriter

A classe `csv.DictWriter` garante a gravação correta de dados baseados em dicionários mapeando chaves para colunas.

```
1 import csv
2
3 campos = ["talhao_id", "area_ha", "cultura"]
4 registros = [
5     {"talhao_id": 1, "area_ha": 120.0, "cultura": "Soja"},
6     {"talhao_id": 2, "area_ha": 85.5, "cultura": "Milho"}
7 ]
8
9 with open("talhoes_mapa.csv", "w", newline="", encoding="utf-8") as f:
10     escritor = csv.DictWriter(f, fieldnames=campos, delimiter=";")
11     escritor.writeheader() # Escreve a linha de cabeçalho com os nomes das
12                             # colunas
13     escritor.writerows(registros)
```

Estruturas em Modo Texto: Módulo json

JSON é o padrão universal para tráfego de dados e configurações semi-estruturadas.

Serialização (json.dump)

```
1 import json
2
3 estacao = {
4     "id": "EST-01",
5     "coords": [-20.94, -48.47],
6     "sensores": ["temp", "umid"],
7     "calibrado": True
8 }
9
10 with open("config.json", "w",
11          encoding="utf-8") as f:
12     json.dump(estacao, f, indent=2)
```

Desserialização (json.load)

```
1 import json
2
3 with open("config.json", "r",
4          encoding="utf-8") as f:
5     dados = json.load(f)
6
7 print("ID:", dados["id"])
8 print("Sensores:", dados["sensores"])
```

O que é um Arquivo Binário?

Definition (Arquivo Binário)

Uma sequência direta de bytes puros (valores entre 0x00 e 0xFF / 0 a 255) sem qualquer decodificação de caracteres ou tradução de quebras de linha.

O Tipo bytes no Python

- Sequência imutável de inteiros de 8 bits.
- Literal de bytes indicado pelo prefixo: `b"Fatec"`.
- Não aceita strings comuns diretamente: exige conversão com `.encode()` ou empacotamento.

Exemplos do Mundo Real

- Imagens de satélite / drone (TIFF, PNG, JPEG).
- Modelos de Machine Learning salvos (.pkl, .onnx).
- Bancos de dados e arquivos compactados (.zip, .sqlite).
- Pacotes de telemetria de rede bruta (CAN bus de tratores).

O Tipo bytearray: Buffers Binários Mutáveis

Enquanto o tipo bytes é imutável, o Python oferece o tipo bytearray para manipulação de bytes *in-place* sem realocação contínua.

```
1 # Criando um buffer mutavel de 5 bytes preenchido com zeros
2 buffer = bytearray(5)
3 print(buffer)  # bytearray(b'\x00\x00\x00\x00\x00')
4
5 # Modificando bytes individuais diretamente por indice
6 buffer[0] = 0x41  # Caractere 'A' em ASCII
7 buffer[1] = 0x47  # Caractere 'G'
8 buffer[2] = 0x52  # Caractere 'R'
9 buffer[3] = 0x4F  # Caractere 'O'
10 buffer[4] = 0x21  # Caractere '!'
11 print(buffer.decode("ascii"))  # 'AGRO!'
12
13 # Gravando o buffer diretamente em um arquivo binario
14 with open("buffer.bin", "wb") as f_bin:
15     f_bin.write(buffer)
```

Modo Texto vs. Modo Binário na Prática

Abrir com 'rb' ou 'wb' desliga toda abstração de codificação de caracteres.

```
1 # Gravacao em modo texto: passamos uma String
2 with open("exemplo_texto.txt", "w", encoding="utf-8") as f:
3     f.write("Ola Big Data")
4
5 # Gravacao em modo binario: DEVEMOS passar Bytes!
6 with open("exemplo_bin.bin", "wb") as f:
7     # f.write("Ola") -> TypeError: a bytes-like object is required, not 'str'
8     dados_bytes = "Ola Big Data".encode("utf-8")
9     f.write(dados_bytes)
10
11 # Leitura em modo binario
12 with open("exemplo_bin.bin", "rb") as f:
13     conteudo_bruto = f.read()
14     print("Tipo lido:", type(conteudo_bruto)) # <class 'bytes'>
15     print("Bytes:", conteudo_bruto)          # b'Ola Big Data'
16     print("Decodificado:", conteudo_bruto.decode("utf-8")) # 'Ola Big Data'
```

Processamento de Arquivos Grandes em Blocos (*Chunking*)

```
1 TAMANHO_BLOCO = 64 * 1024  # 64 KB por bloco
2
3 def clonar_arquivo_grande(origem: str, destino: str) -> None:
4     with open(origem, "rb") as f_in, open(destino, "wb") as f_out:
5         while True:
6             bloco = f_in.read(TAMANHO_BLOCO)
7             if not bloco:
8                 break  # Fim do arquivo (EOF) alcançado
9             f_out.write(bloco)  # Processa/grava bloco a bloco
```

Eficiência Assintótica de Memória

O algoritmo executa com consumo de memória estritamente $O(1)$, independentemente de o arquivo ter 100 MB ou 50 GB.

Verificação de Integridade em Streaming (Hashing SHA-256)

Para certificar que arquivos de imagens de satélite ou dados de telemetria não foram corrompidos na transmissão:

```
1 import hashlib
2
3 def calcular_hash_arquivo(caminho: str) -> str:
4     sha256 = hashlib.sha256()
5     with open(caminho, "rb") as f:
6         while True:
7             bloco = f.read(64 * 1024)
8             if not bloco:
9                 break
10            sha256.update(bloco)  # Atualiza o calculo criptografico com o
11            bloco
12            return sha256.hexdigest()
13
14 # Exemplo:
15 # hash_arquivo = calcular_hash_arquivo("ortomosaico_talhao10.tif")
16 # print("SHA-256:", hash_arquivo)
```

Serialização com o Módulo pickle

O módulo pickle serializa objetos Python arbitrários em fluxos de bytes binários.

Serialização (pickle.dump)

```
1 import pickle
2
3 rede = {
4     "gateway": 9941,
5     "calib": (1.05, 0.98),
6     "nos": ["No-A", "No-B"]
7 }
8
9 with open("rede.pkl", "wb") as f:
10     pickle.dump(rede, f)
```

Desserialização (pickle.load)

```
1 import pickle
2
3 with open("rede.pkl", "rb") as f:
4     rede_restaurada = pickle.load(f)
5
6 print(type(rede_restaurada))    # dict
7 print(rede_restaurada["calib"])
```

Atenção à Segurança com pickle

Risco Crítico de Segurança!

Nunca utilize `pickle.load()` em arquivos ou dados recebidos de fontes não confiáveis ou pela rede!

- O processo de desserialização do `pickle` pode executar código arbitrário na máquina receptora (possibilitando ataques de injeção de comandos / RCE).
- O formato do `pickle` é proprietário do Python (não interoperável com linguagens como Java, C++ ou C#).
- **Alternativas Seguras:**
 - Para troca de dados estruturados com outras plataformas: **JSON**, **Protocol Buffers**, **MessagePack**.
 - Para matrizes numéricas pesadas: formatos dedicados como **HDF5**, **Parquet** ou arquivos binários estruturados com `struct`.

Empacotamento de Tipos com o Módulo struct

O módulo struct permite converter tipos de dados do Python para representações binárias compactas de baixo nível (padrão C).

Códigos de Formato Comuns

- 'i': Inteiro com sinal (4 bytes).
- 'f': Float precisão simples (4 bytes).
- 'd': Double / Float 64 bits (8 bytes).
- 'h': Short int 16 bits (2 bytes).
- '?': Booleano (1 byte).
- '10s': Cadeia de 10 bytes brutos.

Ordem de Bytes (Endianness)

- '<': Little-endian (arquiteturas padrão x86/ARM).
- '>': Big-endian (formato padrão de rede / Network byte order).
- '=': Nativo sem alinhamento de padding.

Exemplo Prático: struct.pack e unpack

struct converte tipos do Python para bytes compactos de baixo nível (padrão C).

Empacotamento e Gravação

```
1 import struct
2
3 fmt = "<ifd"  # int (4B), float (4B),
4           double (8B)
5 tam = struct.calcsize(fmt)  # 16B
6
7 # ID Estacao, Temperatura, Pressao
8
9 pct = struct.pack(fmt, 101, 28.5,
10                    1013.25)
11
12 with open("estacao.bin", "wb") as f:
13     f.write(pct)
```

Leitura e Desempacotamento

```
1 import struct
2
3 fmt = "<ifd"
4 tam = struct.calcsize(fmt)
5
6 with open("estacao.bin", "rb") as f:
7     dados = f.read(tam)
8     s_id, temp, pres = struct.unpack(
9         fmt, dados)
10
11 print(f"ID: {s_id} | T: {temp:.1f} |
12       P: {pres:.1f}")
```

Reconhecimento de Assinatura Binária (*Magic Numbers*)

Arquivos binários reais iniciam com bytes identificadores fixos nos primeiros offsets:

```
1 def identificar_tipo_arquivo(caminho: str) -> str:
2     with open(caminho, "rb") as f:
3         assinatura = f.read(4)
4
5     if assinatura.startswith(b"\x89PNG"):
6         return "Imagem PNG"
7     elif assinatura.startswith(b"%PDF"):
8         return "Documento PDF"
9     elif assinatura.startswith(b"PK\x03\x04"):
10        return "Arquivo Compactado ZIP / DOCX / XLSX"
11    elif assinatura.startswith(b"\xFF\xD8\xFF"):
12        return "Imagem JPEG"
13    return "Formato Binario Desconhecido"
```

Tabela Comparativa de Características

Critério	Modo Texto ('t')	Modo Binário ('b')
Tipo Manipulado	str (cadeia de caracteres Unicode)	bytes / bytearray (sequência de octetos)
Interpretação	Decodifica bytes via encoding (UTF-8, ASCII)	Não há decodificação; leitura de bytes puros
Tradução de Linhas	Converte \n e \r\n automaticamente	Mantém bytes 0x0A e 0x0D intactos
Tamanho em Disco	Maior (um número 1000000 ocupa 7 bytes como texto)	Menor (o mesmo número ocupa 4 bytes como int32)
Legibilidade	Legível diretamente em qualquer editor	Illegível diretamente (requer visualizador hexadecimal)
Velocidade de I/O	Mais lenta (overhead de parsing e conversões)	Ultra-rápida (leitura direta em memória via DMA)

Impacto no Volume de Dados em Escala

Considere 100.000.000 de leituras de telemetria contendo:

- Timestamp inteiro (1725465600)
- ID do Sensor (8540)
- Temperatura float (32.75)
- Umidade float (58.20)

Armazenamento em Modo Texto (CSV)

- Linha:
"1725465600;8540;32.75;58.20\n"
- Tamanho por linha: ≈ 28 bytes
- Total: $\approx$ **2,8** GB
- Overhead de conversão de strings na leitura.

Armazenamento Binário (struct)

- Registro: $\langle \text{Iiff} (4 + 4 + 4 + 4) \rangle$
- Tamanho por linha: **16** bytes fixos
- Total: $\approx$ **1,6** GB (**43%** menor!)
- Leitura direta sem conversão de texto.

Cenários de Aplicação no Agronegócio

Quando usar Modo Texto?

- Relatórios finais para inspeção humana.
- Exportação e importação de planilhas CSV.
- Arquivos de configuração legíveis (.json, .yaml, .ini).
- Logs de auditoria legíveis por administradores.

Quando usar Modo Binário?

- Telemetria massiva de sensores IoT em tempo real.
- Imagens espectrais de safras e ortomosaicos de drones.
- Caching e serialização rápida de modelos de IA.
- Protocolos de comunicação proprietários de tratores (ex: ISOBUS).

Exercício Prático 1: Enunciado

Problema 1: Processamento Streaming de Arquivo de Texto CSV

Uma fazenda monitora 500 sensores de umidade do solo que gravam um arquivo CSV de telemetria com milhões de linhas chamado `sensores_campo.csv` no formato:

```
id_sensor;data_hora;umidade_percentual
```

Escreva um programa Python eficiente que calcule:

- 1 A média geral de umidade de todas as leituras válidas.
- 2 O total de leituras anômalas (onde a umidade foi menor que 30%).
- 3 **Restrição:** O programa deve processar linha a linha sem carregar todo o arquivo na memória RAM.

Exercício Prático 1: Solução Comentada

```
1 def processar_telemetria_streaming(caminho_csv: str) -> None:
2     soma_umidade, total, anomalias = 0.0, 0, 0
3     try:
4         with open(caminho_csv, "r", encoding="utf-8") as f:
5             f.readline() # Descarta cabeçalho
6             for linha in f:
7                 partes = linha.strip().split(";")
8                 if len(partes) == 3:
9                     try:
10                        umid = float(partes[2])
11                        soma_umidade += umid
12                        total += 1
13                        if umid < 30.0:
14                            anomalias += 1
15                    except ValueError:
16                        continue
17
18             if total > 0:
19                 print(f"Total: {total} | Media: {soma_umidade/total:.2f}% | Anomalias: {
20 anomalias}")
21     except FileNotFoundError:
22         print("Arquivo nao encontrado.")
```

Exercício Prático 2: Enunciado

Problema 2: Arquivo Binário com Estrutura e Cabeçalho Fixo

Projete um sistema de armazenamento binário para dados de estações meteorológicas:

- Cada registro possui tamanho fixo de ****12 bytes****:
 - ① ID do Sensor (int de 4 bytes)
 - ② Temperatura (float de 4 bytes)
 - ③ Umidade (float de 4 bytes)
- Crie uma função para gravar uma lista de registros em arquivo binário usando o módulo `struct`.
- Crie uma função que faça acesso direto (*Random Access*) ao k -ésimo registro usando `seek()` sem precisar ler os registros anteriores.

Exercício Prático 2: Solução Comentada (Gravação)

```
1 import struct
2
3 FORMATO = "<iff" # Little-endian: int (4B), float (4B), float (4B)
4 TAM_REG = struct.calcsize(FORMATO) # 12 bytes
5
6 def gravar_registros_binarios(nome_arquivo: str, registros: list) -> None:
7     with open(nome_arquivo, "wb") as f_bin:
8         for sensor_id, temp, umid in registros:
9             pacote = struct.pack(FORMATO, sensor_id, temp, umid)
10            f_bin.write(pacote)
11        print(f"Gravados {len(registros)} registros ({len(registros) * TAM_REG} bytes).")
12
13 # Dados de teste
14 amostras = [
15     (101, 24.5, 65.0),
16     (102, 28.2, 42.1),
17     (103, 31.0, 38.5),
18     (104, 22.8, 70.2)
19 ]
20 gravar_registros_binarios("estacoes.dat", amostras)
```

Exercício Prático 2: Solução Comentada (Acesso Direto $O(1)$)

```
1 def ler_registro_por_indice(nome_arquivo: str, indice: int):
2     # Busca o k-esimo registro diretamente via seek em tempo  $O(1)$ 
3     try:
4         with open(nome_arquivo, "rb") as f_bin:
5             f_bin.seek(indice * TAM_REG) # Salto direto!
6             bloco = f_bin.read(TAM_REG)
7             if len(bloco) < TAM_REG:
8                 return None
9             return struct.unpack(FORMATO, bloco)
10    except FileNotFoundError:
11        return None
12
13 # Busca instantanea do 3o registro (indice 2) sem ler os anteriores!
14 registro = ler_registro_por_indice("estacoes.dat", 2)
15 if registro:
16     s_id, temp, umid = registro
17     print(f"Registro [2]: ID={s_id} | Temp={temp:.1f} C | Umid={umid:.1f}%")
```

Exercício Prático 3: Enunciado

Problema 3: Pipeline de Conversão Texto → Binário Otimizado

Escreva um programa Python que execute um pipeline de compactação de telemetria:

- 1 Leia um arquivo de texto CSV de telemetria linha a linha (*streaming*).
- 2 Filtre apenas as leituras válidas (temperatura entre -10°C e 60°C).
- 3 Grave os registros validados diretamente em um arquivo binário estruturado com `struct`.
- 4 Exiba na tela a redução percentual de espaço em disco alcançada.

Exercício Prático 3: Solução Comentada

```
1 import os, struct
2
3 def converter_csv_para_binario(arquivo_csv: str, arquivo_bin: str) -> None:
4     fmt = "<if" # ID Sensor (int 4B), Temperatura (float 4B) = 8 bytes
5     with open(arquivo_csv, "r", encoding="utf-8") as f_in, open(arquivo_bin, "wb")
6         as f_out:
7         for linha in f_in:
8             partes = linha.strip().split(";")
9             if len(partes) >= 2:
10                 try:
11                     s_id, temp = int(partes[0]), float(partes[1])
12                     if -10.0 <= temp <= 60.0:
13                         f_out.write(struct.pack(fmt, s_id, temp))
14                 except ValueError:
15                     continue
16
17     tam_txt, tam_bin = os.path.getsize(arquivo_csv), os.path.getsize(arquivo_bin)
18     print(f"Texto: {tam_txt}B | Binario: {tam_bin}B | Reducao: {(1 - tam_bin/tam_txt)
19     }*100:.1f}%")
```


Próximos Passos

Conteúdo da Próxima Aula (Aula 08)

- **Algoritmos de Busca: Busca Sequencial (Linear Search)**
- Princípios de varredura em estruturas lineares e arquivos.
- Análise de complexidade no melhor, pior e caso médio ($O(1)$ e $O(n)$).
- Otimizações com sentinela e busca em arquivos indexados.

Tarefa Complementar

Implementar um script que leia um arquivo de log binário de telemetria, converta para CSV e compare a diferença de tamanho em disco gerada entre os dois formatos.

Bons Estudos!

Dúvidas ou Perguntas?